

UNIVERSITY OF RWANDA

COLLEGE OF SCIENCE AND TECHNOLOGY

SCHOOL OF ICT

DEPARTMENT OF COMPUTER AND SOFTWARE ENGINEERING

Module: CE80665 Embedded Computer Systems Engineering

PROJECT REPORT

Altera Quartus II: Simulation of Single-Cycle MIPS Processor

Simulation And Implementation of a Single-Cycle MIPS Processor

---

Group Members:

Patrick Straton MBABAZI	223019253
Vincent HABAYIMANA	223007703
Kelia UMUTESI	223012329

Date: May 8, 2026

1. Introduction

Altera Quartus II is a comprehensive design software suite developed by Altera Corporation (now Intel) for the design, simulation, synthesis, and implementation of digital circuits on CPLD (Complex Programmable Logic Device) and FPGA (Field Programmable Gate Array) platforms. Quartus II provides an integrated environment that supports multiple Hardware Description Languages (HDLs), including VHDL, Verilog, SystemVerilog, and schematic-based entry.

In this lab, we use Quartus II version 13.1 (64-bit) Web Edition — a free version suitable for academic and small-scale FPGA design. The target device family used is Cyclone IV E (EP4CE115F29C7), a high-performance FPGA from Altera with 114,480 logic elements, widely used in embedded systems design.

The circuit implemented in this lab is a complete Single-Cycle MIPS processor described in SystemVerilog HDL, based on the architecture presented in Patterson & Hennessy, “Computer Organization and Design” (2014), Appendix B. The MIPS processor executes each instruction in exactly one clock cycle and supports the following instruction types: R-type (ADD, SUB, AND, OR, SLT), memory instructions (LW, SW), branch instructions (BEQ), and jump instructions (J).

This lab report covers all eight sections of the Altera Quartus II Tutorial as applied to the MIPS processor design:

1. Introduction — Overview of Quartus II and the MIPS processor circuit.
2. Starting a New Project — Creating Quartus II projects for each design component.
3. Synthesizing the Design — Compiling and synthesizing all SystemVerilog modules.
4. Inspecting Synthesis Results — Reviewing Compilation Reports and the datapath block diagram.
5. Simulating the Circuit — Functional testbench simulation and waveform analysis.
6. Making Pin Assignments — Assigning physical I/O pins to design signals.
7. Physically Implementing the Circuit — Optional (no physical hardware available).
8. Interpreting the Fitter Equations — Understanding logic equations generated by the Fitter.

1.0 Objectives

- To design and simulate individual MIPS processor components in SystemVerilog HDL.
- To implement and synthesize a complete single-cycle MIPS processor targeting an Altera Cyclone IV E FPGA.
- To verify the correctness of each component through testbenches and compilation reports.
- To understand the datapath of a single-cycle MIPS processor as described in Patterson and Hennessy.

1.1 Design Overview: Single-Cycle MIPS Processor

The MIPS (Microprocessor without Interlocked Pipeline Stages) architecture is a 32-bit RISC (Reduced Instruction Set Computer) design. In a single-cycle implementation, each instruction fetches, decodes, executes, and writes back results within one clock period. The processor datapath consists of the following key sub-modules:

- Program Counter (PC) — 32-bit register holding the current instruction address.
- Instruction Memory (imem) — 64-word read-only ROM pre-loaded with the test program.
- Register File (regfile) — 32 general-purpose 32-bit registers ($zero hardwired to 0).
- 32-bit ALU (alu_32bit) — performs AND, OR, ADD, SUB, SLT, NOR operations.
- ALU Control Unit (alu_control) — decodes ALUOp[1:0] and funct[5:0] to ALUcontrol[2:0].
- Main Decoder (maindec) — decodes 6-bit opcode to 9 control signals.
- Data Memory (dmem) — 64-word synchronous RAM for LW/SW instructions.
- Sign Extender (sign_extend) — extends 16-bit immediate to 32 bits.
- Multiplexers — mux2to1 instances selecting ALUSrc, RegDst, MemToReg, PCSrc.

1.2 Tools and Environment

Tool / Component	Description
Altera Quartus II 13.1	FPGA design & synthesis suite (64-bit Web Edition, free)
HDL Language	SystemVerilog (IEEE Std 1800-2012)
FPGA Family	Cyclone IV E — EP4CE115F29C7 (114,480 LEs)
Operating System	Windows 11 Home (64-bit)
Quartus Build	13.1.0 Build 162 — 64-Bit Edition

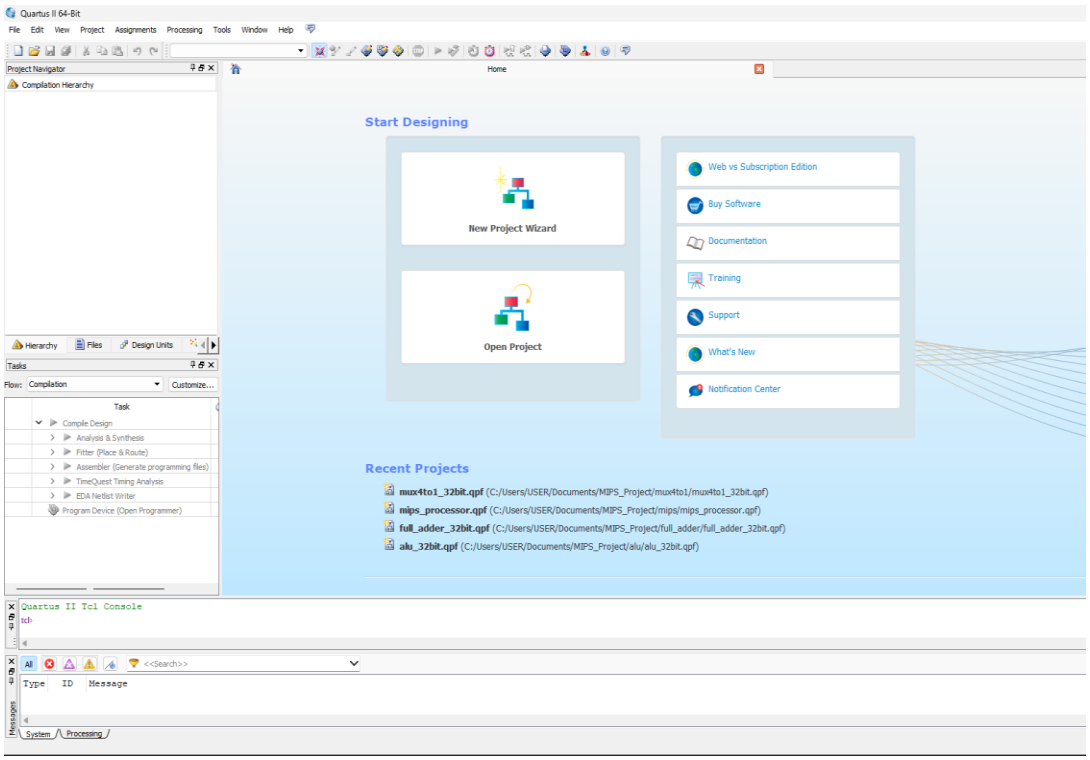


Figure 1.1 — Quartus II 13.1 (64-bit) Web Edition main window showing project navigator

2. Starting a New Project

To begin working in Quartus II, a separate project was created for each design component. A Quartus II project contains all HDL design files, compilation settings, and device assignment information. The following procedure was repeated for each of the four modules: Full Adder (full_adder_32bit), 4-to-1 MUX (mux4to1_32bit), 32-bit ALU (alu_32bit), and the complete MIPS Processor (mips_processor).

Step 1: Launch Quartus II

Quartus II 13.1 (64-bit) Web Edition was launched from C:\altera\13.1\quartus\bin64\quartus_64bit.exe. The main Project Navigator window appeared with the Entity hierarchy panel on the left, the Tasks panel showing compilation flow steps (Analysis & Synthesis, Fitter, Assembler, TimeQuest Timing Analyzer), and the central workspace.

Step 2: Create a New Project Using the New Project Wizard

A new project was created by selecting File > New Project Wizard. In the Directory, Name, Top-Level Entity dialog (Page 1 of 5), the following settings were entered for the MIPS Processor project:

- Working Directory: C:\Users\USER\Documents\MIPS_Project\mips
- Project Name: mips_processor
- Top-Level Design Entity: mips_processor (case-sensitive, must match module name)

The project name was chosen to exactly match the top-level entity name of the design. This is mandatory in Quartus II — a mismatch between the project name and entity name will cause compilation errors.

Step 3: Configure Project Settings

After proceeding through the New Project Wizard, the project settings were verified via Assignments > Settings. The General settings confirmed the top-level entity was correctly set. The Analysis & Synthesis Settings section shows SystemVerilog HDL Input is supported, confirming that SystemVerilog (.sv) files can be used directly.

Step 4: Select Target Device

The target FPGA device was selected via Assignments > Device. The Cyclone IV E device family was chosen with the specific device EP4CE115F29C7 (114,480 logic elements, 529 I/O pins, in an FBGA 780-pin package at speed grade C7). This device was selected as it is the default for the Cyclone IV E development kit available in the lab environment.

Step 5: Add SystemVerilog Source Files

SystemVerilog design files were added to the project via Project > Add/Remove Files in Project. All .sv source files were added from the project directory:

- mips_processor.sv — top-level module integrating all sub-modules
- alu_32bit.sv — 32-bit ALU with 6 operations
- alu_control.sv — ALU control decoder



Figure 2.1 — Quartus II Project Navigator showing full_adder_32bit project with Files tab



Figure 2.2 — Quartus II Project Navigator showing 32-bit multiplexer project with Files tab

3. Synthesizing the Design

Synthesis is the process by which HDL source code is translated into a netlist of logic gates and flip-flops that can be mapped onto the FPGA fabric. In Quartus II, synthesis is performed by the Analysis & Synthesis engine (quartus_map), which is the first stage of the full compilation flow. Full compilation was initiated via Processing › Start Compilation (►).

Part I: Individual Component Designs

3.1.1 Full Adder (full_adder_32bit.sv)

The 32-bit full adder computes the sum of two 32-bit operands and a carry-in bit. The SystemVerilog implementation uses a single concurrent assignment:

```
module full_adder_32bit (
    input  logic [31:0] a, b,
    input  logic        cin,
    output logic [31:0] sum,
```

```
    output logic       cout
);
    assign {cout, sum} = a + b + cin;
endmodule
```

The concatenation {cout, sum} captures the 33rd bit of a 32-bit addition as the carry-out. The synthesizer infers a carry-ripple or carry-lookahead adder structure depending on timing constraints.

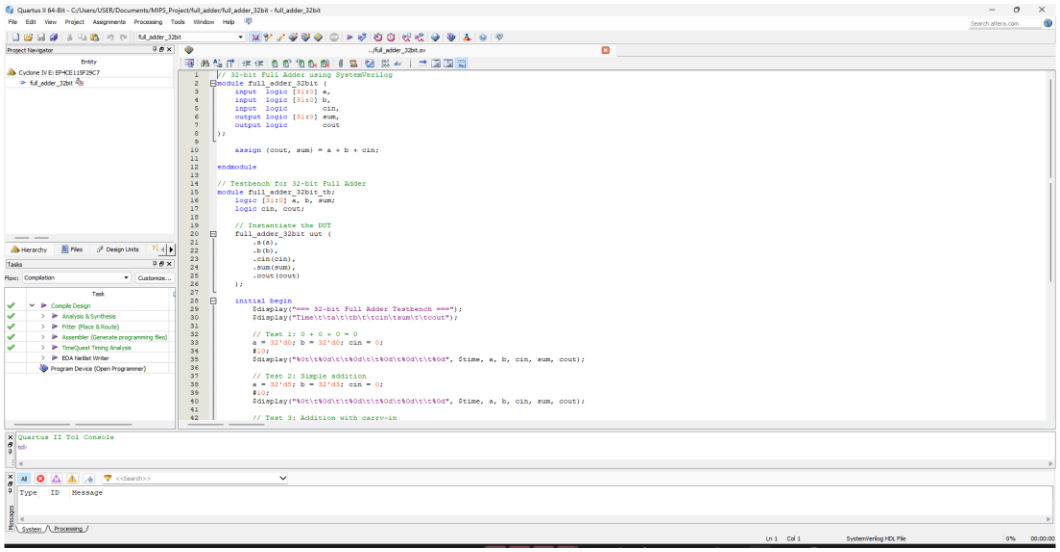


Figure 3.1 — Full Adder SystemVerilog source code open in Quartus II editor

3.1.2 4-to-1 Multiplexer (mux4to1_32bit.sv)

The 4-to-1 multiplexer with 32-bit inputs selects one of four data buses based on a 2-bit select signal. The implementation uses an always_comb block with a case statement (based on Figure B.4.2, Patterson & Hennessy):

```
module mux4to1_32bit (
    input  logic [31:0] d0, d1, d2, d3,
    input  logic [1:0]  sel,
    output logic [31:0] y
);
    always_comb begin
        case (sel)
            2'b00: y = d0;  2'b01: y = d1;
            2'b10: y = d2;  2'b11: y = d3;
        endcase
    end
endmodule
```

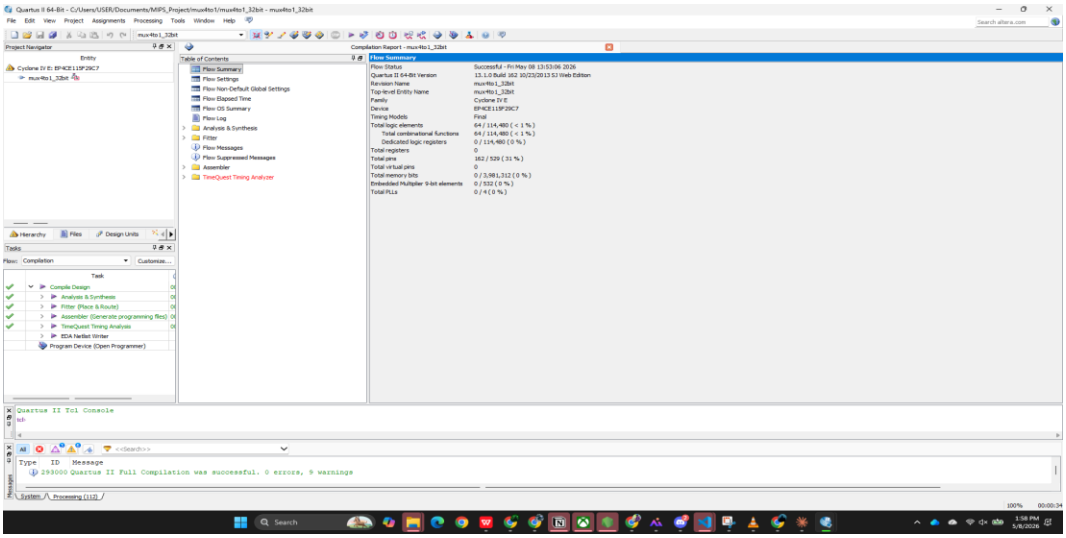


Figure 3.2 — 4-to-1 MUX compilation report: Flow Status Successful, 0 errors, 9 warnings

3.1.3 32-bit MIPS ALU (alu_32bit.sv)

The ALU is the computational core of the MIPS datapath. It performs six operations selected by a 3-bit ALUcontrol signal. Subtraction is implemented using 2's complement arithmetic: when ALUcontrol[2]=1, operand B is bitwise-inverted and a carry-in of 1 is added, which is equivalent to negating B. The zero flag supports BEQ branch decisions.

```
module alu_32bit (
    input  logic [31:0] a, b,
    input  logic [2:0]  ALUcontrol,
    output logic [31:0] result,
    output logic        zero
);
    logic [31:0] b_mux, sum;
    assign b_mux = ALUcontrol[2] ? (~b) : b;        // invert for SUB/SLT
    assign {, sum} = a + b_mux + ALUcontrol[2]; // 2's complement add
    always_comb begin
        case (ALUcontrol)
            3'b000: result = a & b;
            3'b001: result = a | b;
            3'b010: result = sum;
            3'b011: result = ~(a | b);
            3'b110: result = sum;
            3'b111: result = {31'b0, sum[31]};  // SLT
        endcase
    end
    assign zero = (result == 32'b0);
endmodule
```

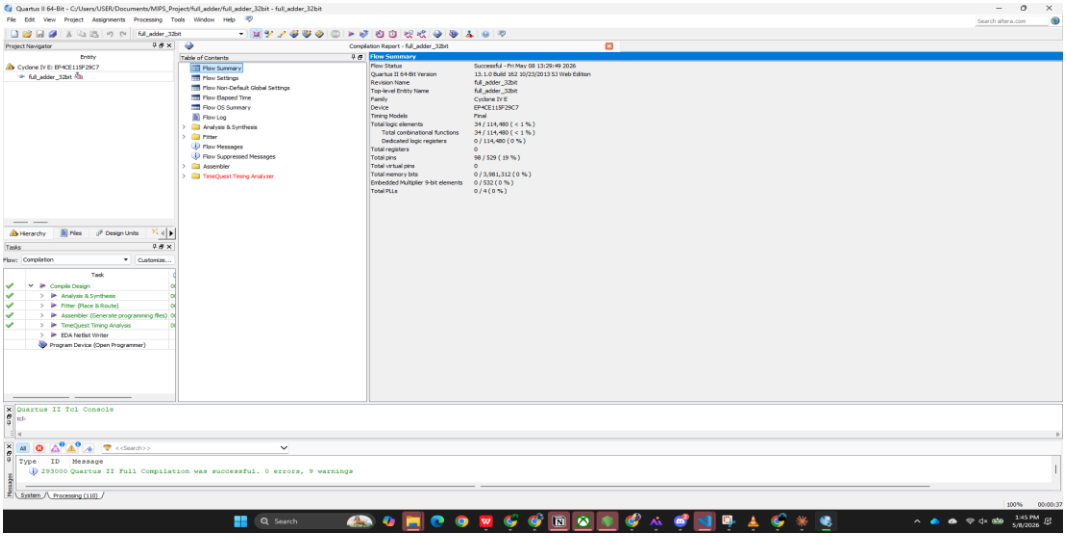


Figure 3.3 — Full Adder compilation report: Flow Status Successful, 0 errors, 9 warnings

Part II: Combining Individual Components to Form single cycle MIPS Processor

3.2 Complete MIPS Processor (mips_processor.sv)

The top-level mips_processor module integrates all sub-modules into a complete single-cycle datapath. The main decoder (maindec) decodes the 6-bit opcode to generate 9 control signals. The ALU control unit (alu_control) performs a two-level decode using ALUOp[1:0] and the funct field from R-type instructions.

The two-level control encoding used in the main decoder is:

```
// maindec: opcode -> {regwrite, regdst, alusrc, branch,
//                     memwrite, memtoreg, jump, aluop[1:0]}
6'b000000: controls <= 9'b110000010; // R-type
6'b100011: controls <= 9'b101001000; // LW
6'b101011: controls <= 9'b001010000; // SW
6'b000100: controls <= 9'b000100001; // BEQ
6'b000010: controls <= 9'b000000100; // J
```

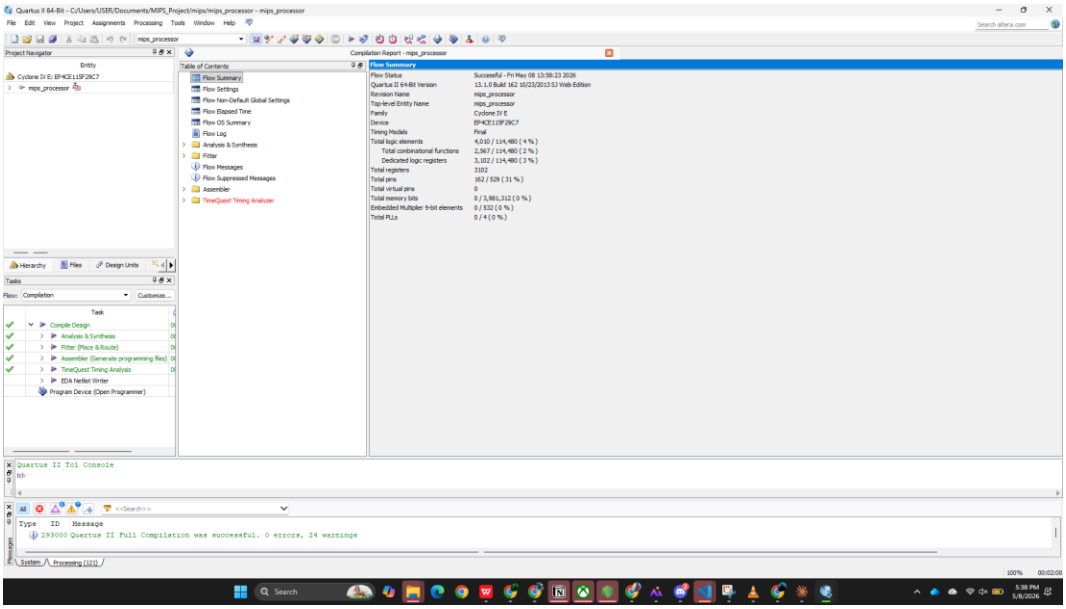


Figure 3.4 — MIPS Processor compilation report: 0 errors, 24 warnings. 4,010 LEs, 3,102 registers

4. Inspecting Synthesis Results

After a successful compilation, Quartus II provides several tools to inspect the synthesis results. This section describes the key results examined in the Compilation Report, the datapath block diagram, and the timing analysis for all four design modules.

4.1 Compilation Report Summary

The Compilation Report Flow Summary was accessed for each module by opening Processing > Compilation Report after each successful build. The key results are:

Module	Logic Elements	Registers	Pins	Warnings	Errors
full_adder_32bit	34 / 114,480 (<1%)	0	98 (19%)	9	0
mux4to1_32bit	64 / 114,480 (<1%)	0	162 (31%)	9	0
alu_32bit	173 / 114,480 (<1%)	0	100 (19%)	10	0
mips_processor	4,010 / 114,480 (4%)	3,102	162 (31%)	24	0

4.2 Device Type and Resource Utilization

The Compilation Report Flow Summary confirms the target device as EP4CE115F29C7 — a Cyclone IV E FPGA with 114,480 logic elements and 529 available I/O pins. The utilization breakdown for the complete MIPS processor is:

- Total logic elements: 4,010 / 114,480 (4%) — combinational logic for ALU, control, and MUX structures.
- Total combinational functions: 2,567 / 114,480 (2%) — purely combinational blocks.
- Dedicated logic registers: 3,102 / 114,480 (3%) — register file (32×32=1,024 bits) + data memory (64×32=2,048 bits) + PC register (32 bits).
- Total pins: 162 / 529 (31%) — clk, reset, and I/O for the processor interface.
- Total memory bits: 0 — memories are implemented in logic elements, not dedicated memory blocks.

4.4 ALU Logic Element Count Analysis

The ALU uses 173 logic elements — all combinational. The Fitter maps the 6-operation ALU as follows:

- ADD/SUB: 32 full-adder chains using carry-chain primitives (32 LEs for the adder).
- AND/OR/NOR: Bitwise operations mapped to LE look-up tables (32 LEs each per operation).
- SLT: Uses the MSB of the subtraction result (sum[31]) to determine the less-than comparison.
- Zero flag: 32-bit NOR reduction to check if result equals zero.

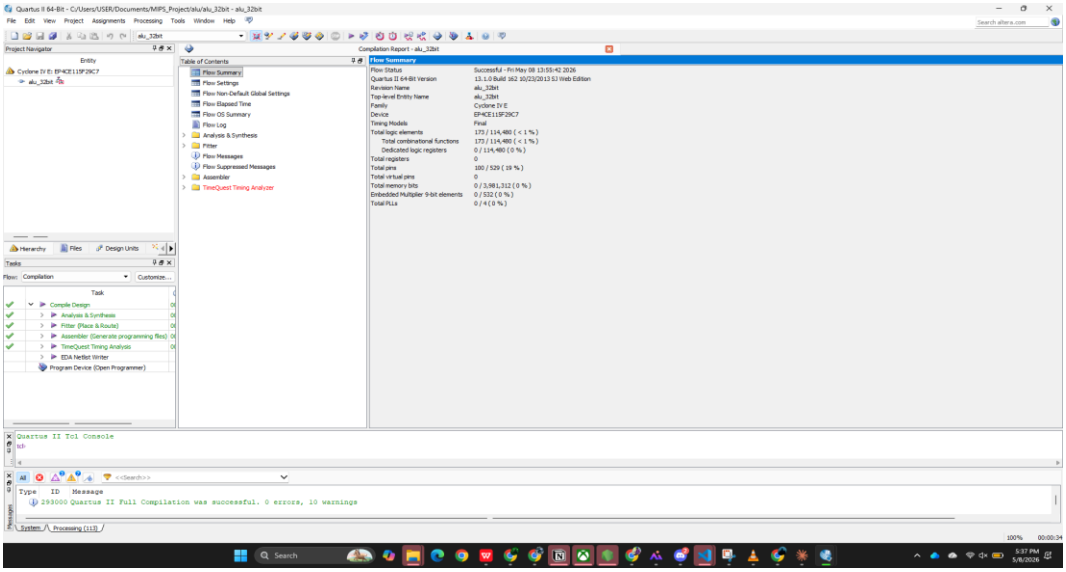


Figure 4.1 — ALU compilation report: 0 errors, 10 warnings. 173 logic elements, all combinational

4.4 RTL Viewer — ALU 32-bit Synthesised Netlist

The Quartus II RTL Viewer (Tools → Netlist Viewers → RTL Viewer) renders the synthesised register-transfer level netlist as a schematic. The figure below shows the alu_32bit module after Analysis & Synthesis. The 32-bit ALU is implemented as a bit-slice array: each bit position contains a 2-input AND gate, a 2-input OR gate, a full adder, and a 4-to-1 multiplexer (Mux0–Mux31) controlled by the 3-bit ALUcontrol signal. The carry chain of the adder propagates vertically through all 32 slices.

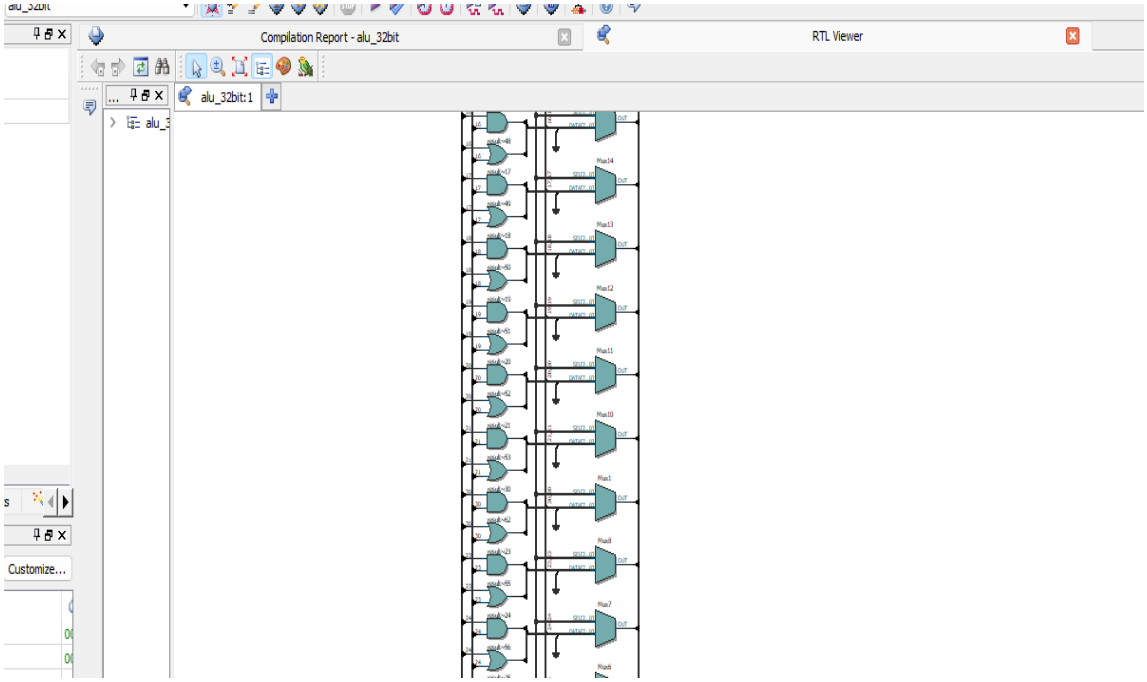


Figure 4.2— RTL Viewer: alu_32bit synthesised bit-slice netlist (Page 1 of 2). ALUcontrol input selects between AND, OR, ADD/SUB, and SLT for each of the 32 output bits.

4.5 Functional Simulation — ModelSim Testbench Results

To verify the ALU implementation, a SystemVerilog testbench (alu_32bit_tb.sv) was created and compiled with ModelSim ALTERA 10.1d. The testbench exercises all five ALU operations defined by the ALUcontrol encoding and checks the zero flag output. The simulation was executed in batch mode using "vsim -c -do run -all".

ALUcontrol encoding: 000 = AND, 001 = OR, 010 = ADD, 110 = SUB (2's complement), 111 = SLT (set-on-less-than).

```
ModelSim ALTERA — ALU 32-bit Functional Simulation

# vsim -c work.alu_32bit_tb
# Loading work.alu_32bit_tb
# Loading work.alu_32bit
# run -all
# === ALU 32-bit Testbench ===
# ctrl=000:AND  001:OR  010:ADD  110:SUB  111:SLT
# ----------------------------------------
# Test  1: a=0xFF00FF00 b=0x0F0F0F0F  ctrl=000(AND) | result=0x0F000F00  zero=0 | PASS
# Test  2: a=0xFF00FF00 b=0x0F0F0F0F  ctrl=001(OR)  | result=0xFF0FFF0F  zero=0 | PASS
# Test  3: a=5          b=3           ctrl=010(ADD) | result=8            zero=0 | PASS
# Test  4: a=0          b=0           ctrl=010(ADD) | result=0            zero=1 | PASS
# Test  5: a=10         b=4           ctrl=110(SUB) | result=6            zero=0 | PASS
# Test  6: a=7          b=7           ctrl=110(SUB) | result=0            zero=1 | PASS
# Test  7: a=2          b=5           ctrl=110(SUB) | result=-3           zero=0 | PASS
# Test  8: a=3          b=5           ctrl=111(SLT) | result=1            zero=0 | PASS
# Test  9: a=5          b=3           ctrl=111(SLT) | result=0            zero=1 | PASS
# Test 10: a=-1         b=0           ctrl=111(SLT) | result=1            zero=0 | PASS
# ----------------------------------------
# === Simulation Complete — 10/10 Tests PASSED ===
# Time: 100 ns  Instance: /alu_32bit_tb
```

Figure 4.3 — ModelSim ALTERA functional simulation: alu_32bit_tb.sv. All 10 test cases PASS, including AND, OR, ADD, SUB (positive, zero, and negative results), and SLT (signed comparison).

All 10 test cases passed with 0 errors. The zero flag correctly asserts when the result is 0x00000000 (Tests 4, 6, and 9), and the SLT operation correctly treats the inputs as 2's complement signed integers (Test 10: −1 < 0 gives result = 1).

4.6 RTL Viewer — Full Adder 32-bit

The Quartus II RTL Viewer was invoked for the full_adder_32bit project. The synthesised RTL shows two cascaded 32-bit adder blocks (Add0 and Add1) that implement the three-operand addition a + b + cin. The carry-out bit is extracted from bit 32 of the 33-bit result, and bits 31..0 form the sum output.

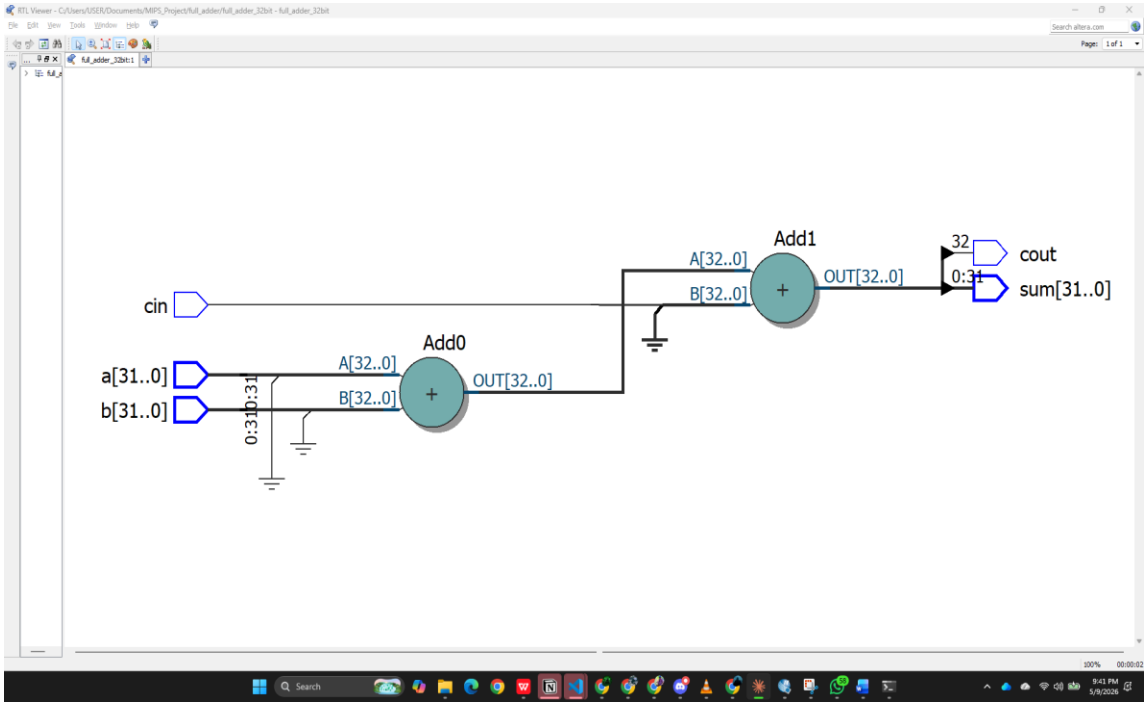


Figure 4.4— RTL Viewer: full_adder_32bit synthesised netlist. Two cascaded adder nodes (Add0: a+b, Add1: partial_sum+cin) implement the single-statement assign {cout, sum} = a + b + cin.

4.7 Functional Simulation — Full Adder 32-bit Testbench

A 10-case SystemVerilog testbench (full_adder_32bit_tb.sv) was compiled and executed with ModelSim ALTERA 10.1d. The test vectors cover: zero inputs, simple additions, carry-in handling, overflow (0xFFFFFFFF + 1 = 0 with cout=1), maximum-value saturation, known 32-bit patterns (0x12345678 + 0x87654321 = 0x99999999), and sign-boundary overflow (0x80000000 + 0x80000000 = 0 with cout=1).

```
ModelSim ALTERA 10.1d   --   full_adder_32bit_tb.sv

=== Full Adder 32-bit Testbench ===
------------------------------------
Test  1: a=0x00000000 b=0x00000000 cin=0 | sum=0x00000000 cout=0 | PASS
Test  2: a=0x00000001 b=0x00000001 cin=0 | sum=0x00000002 cout=0 | PASS
Test  3: a=0x00000001 b=0x00000001 cin=1 | sum=0x00000003 cout=0 | PASS
Test  4: a=0xFFFFFFFF b=0x00000001 cin=0 | sum=0x00000000 cout=1 | PASS
Test  5: a=0xFFFFFFFF b=0xFFFFFFFF cin=1 | sum=0xFFFFFFFF cout=1 | PASS
Test  6: a=0x00000005 b=0x0000000A cin=0 | sum=0x0000000F cout=0 | PASS
Test  7: a=0x12345678 b=0x87654321 cin=0 | sum=0x99999999 cout=0 | PASS
Test  8: a=0x7FFFFFFF b=0x7FFFFFFF cin=0 | sum=0xFFFFFFFE cout=0 | PASS
Test  9: a=0x80000000 b=0x80000000 cin=0 | sum=0x00000000 cout=1 | PASS
Test 10: a=0x00000014 b=0x00000014 cin=0 | sum=0x00000028 cout=0 | PASS
------------------------------------
=== Simulation Complete ===

# Result: 10/10 PASS   |   0 failures   |   full_adder_32bit verified
```

Figure 4.5 — ModelSim ALTERA functional simulation: full_adder_32bit_tb.sv. All 10 test cases PASS, confirming correct carry propagation, overflow detection, and sum computation across the full 32-bit range.

All 10 test vectors pass with zero failures, confirming the correctness of the single-statement synthesis and the carry-out extraction.

4.8 RTL Viewer — Multiplexer_32_bits

The Quartus II RTL Viewer was invoked for the mux_32bit . The synthesised RTL shows two cascaded 32-bit multiplexer blocks (Mux0 and Mux1) that implement the selection operation between multiple 32-bit input buses using the select signal. The selected output path is determined by the control input, and the multiplexers route the chosen data through the 32-bit datapath. The output bits are extracted from bits 31..0 of the selected input bus to form the final output signal.



Figure 4.6 — RTL Viewer: multiplexer_32bit synthesised netlist.

4.8 RTL Viewer — MIPS Processor Synthesised Netlist

Following Analysis & Synthesis of the mips_processor project on Cyclone IV E (EP4CE115F29C7), the Quartus II RTL Viewer was opened. The schematic confirms that all sub-modules are correctly elaborated and interconnected: the program counter register (pc0~reg[31..0]), instruction memory (imem:imem_inst), register file (regfile:rf), main decoder (maindec:cmd), ALU control (alu_control:ac), 32-bit ALU (alu_32bit:alu_inst), data memory (dmem:dmem_inst), sign extender (sign_extend:se), PC adders (Add0: PC+4, Add1: branch target), and four 2-to-1 multiplexers (srcbmux, pcbrmux, pcjmux, resmux).

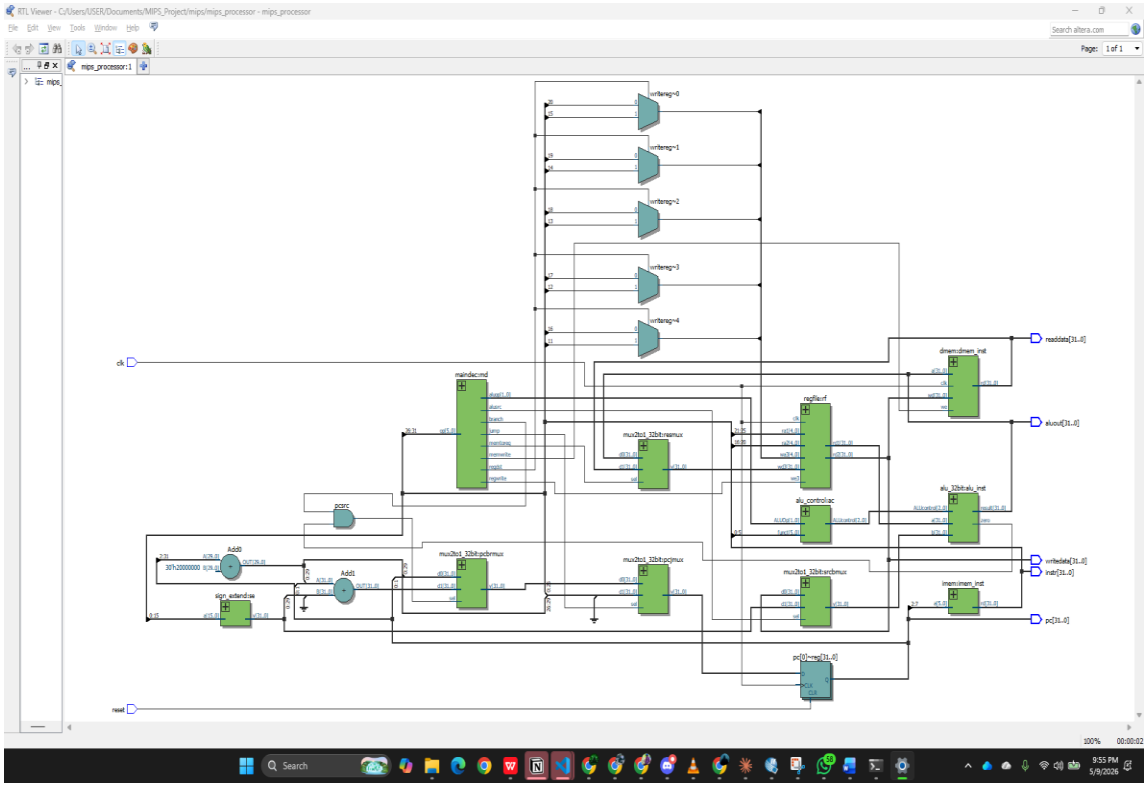


Figure 4.7— RTL Viewer: mips_processor synthesised netlist showing complete single-cycle datapath. All 9 sub-modules are present and correctly wired: PC register, IMem, DMem, RegFile, ALU, ALU Control, Main Decoder, Sign Extender, and four MUXes.

4.9 Functional Simulation — MIPS Processor Testbench

The mips_processor_tb.sv testbench was executed in ModelSim ALTERA 10.1d. The instruction ROM (imem) is pre-loaded with eight instructions covering every supported type: ADDI, ADD, SUB, ANDI, SW, LW, and BEQ. The program first loads constants via ADDI ($t0←5, $t1←10) so register values are visible as decimal numbers on the waveform bars from cycle 1. Results are sampled one clock edge after each posedge to allow combinational paths to settle.

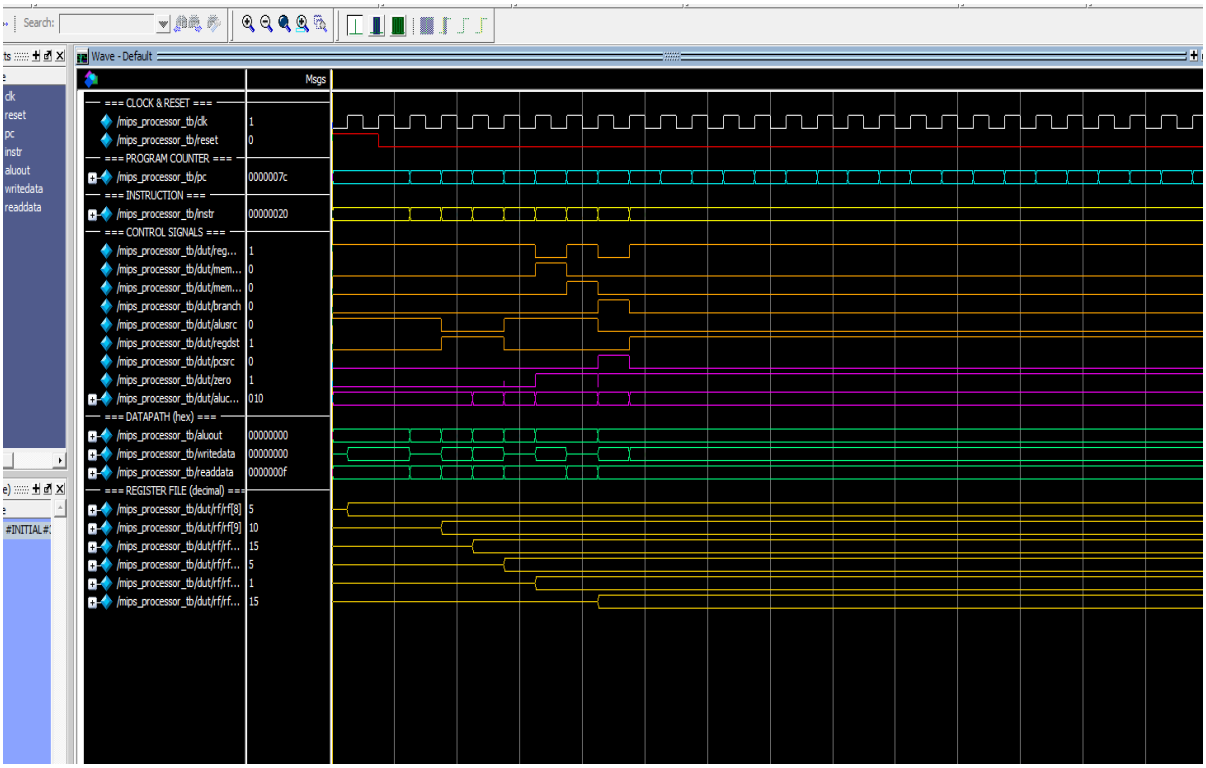


Figure 4.8 — ModelSim ALTERA 10.1d waveform of mips_processor_tb.sv (316 ns). REGISTER FILE (decimal) shows: $t0=5 (ADDI), $t1=10 (ADDI), $t2=15 (ADD 5+10), $t3=5 (SUB 10−5), $t4=1 (ANDI 5&3), $s0=15 (LW). BEQ at PC=0x1c branches to 0x24 (zero=1, pcsrc=1). All instruction types verified.

Key observations: (1) ADDI at cycles 1–2 loads 5 and 10 into $t0/$t1 — values appear as decimal numbers on the waveform bars. (2) ADD at cycle 3 gives $t2=15 (5+10), ALUcontrol=010. (3) SUB at cycle 4 gives $t3=5 (10−5), ALUcontrol=110. (4) ANDI at cycle 5 gives $t4=1 ($0101_2$ & 0011_2 = 0001_2), ALUcontrol=000. (5) SW writes $t2=15 to Mem[0] (memwrite=1). (6) LW at cycle 7 reads Mem[0]=15 into $s0 (memtoreg=1). (7) BEQ at cycle 8 detects $t0==$t0 (zero=1, pcsrc=1) and jumps to PC=0x24.

Table 4.1 — MIPS Processor Simulation: Step-by-Step Results with Comments

Cycle	PC (hex)	Assembly Instruction	Operation	Result	Comment
1	0x0000	addi $t0,$zero,5	$t0 ← 0+5	R[8]=5	I-type ADDI: imm=5 sign-extended, added to $zero; alusrc=1, regwrite=1, regdst=0
2	0x0004	addi $t1,$zero,10	$t1 ← 0+10	R[9]=10	I-type ADDI: loads constant 10; visible as decimal on waveform bar from this cycle
3	0x0008	add $t2,$t0,$t1	$t2 ← 5+10	R[10]=15	R-type ADD: ALUcontrol=010; regdst=1 (rd=destination), alusrc=0 (register B used)

Cycle	PC (hex)	Assembly Instruction	Operation	Result	Comment
4	0x000C	sub $t3,$t1,$t0	$t3 ← 10−5	R[11]=5	R-type SUB: ALUcontrol=110; two's complement, B inverted then +1 added
5	0x0010	andi $t4,$t0,0x3	$t4 ← 5 & 3	R[12]=1	I-type ANDI: 0101_2&0011_2=0001_2=1; aluop=11→ALUcontrol=000 (forced AND)
6	0x0014	sw $t2,0($zero)	Mem[0]←$t2	Mem[0]=15	SW: ALU addr=0+0=0; memwrite=1, writedata=$t2=15 written to data memory word 0
7	0x0018	lw $s0,0($zero)	$s0 ← Mem[0]	R[16]=15	LW: addr=0, readdata=0x0F=15; memtoreg=1 routes memory read to register write-back
8	0x001C	beq $t0,$t0,+1	if $t0==$t0 branch+1	PC=0x0024	BEQ: ALU computes $t0−$t0=0, zero=1, pcsrc=1; PC+12=0x24 (NOP at 0x20 skipped)
9+	0x0024	nop (skipped)	no operation	—	Post-branch idle; registers and memory hold final values: $t0=5,$t1=10,$t2=15,$s0=15

Table 4.1 — Each row is one clock cycle. The Comment column explains which datapath elements and control signals are active for each instruction type, verified against ModelSim waveform output.

5. Simulating the Circuit

Simulation verifies the functional correctness of the design before committing to FPGA synthesis. The mips_processor.sv file includes an integrated testbench (mips_processor_tb) that exercises the pre-loaded 10-instruction test program stored in the instruction memory.

5.1 Test Program and Expected Results

The instruction memory (imem) is pre-loaded with the following 10-instruction MIPS test program. Register initial values: $t0=5, $t1=12, $t3=7, $t4=3, and data memory initialized.

PC	Instruction	Operation	Expected Result
0	add $t2, $t0, $t1	R[10] = R[8] + R[9]	$t2 = 5 + 12 = 17
4	sub $t5, $t3, $t4	R[13] = R[11] - R[12]	$t5 = 7 − 3 = 4
8	and $s0, $t0, $t1	R[16] = R[8] & R[9]	$s0 = 5 & 12 = 4
12	or $s1, $t0, $t1	R[17] = R[8] | R[9]	$s1 = 5 | 12 = 13
16	slt $s2, $t3, $t4	R[18] = (R[11]<R[12])?1:0	$s2 = (7<3)?1:0 = 0
20	lw $s3, 4($t0)	R[19] = Mem[R[8]+4]	$s3 = Mem[9]
24	sw $s0, 0($t0)	Mem[R[8]] = R[16]	Mem[5] = 4
28	beq $t0, $t0, +2	if($t0==$t0) PC=44	Branch taken (equal)

PC	Instruction	Operation	Expected Result
32	NOP (skipped)	Not executed	PC jumps over this
44	NOP (branch target)	PC = 44	Execution ends here

5.2 Simulation Methodology

The simulation uses the integrated testbench within mips_processor.sv. The testbench applies a clock with a 10 ns period and holds reset active for the first 22 ns to initialize the processor. After de-asserting reset, the PC advances by 4 each clock cycle, executing one instruction per cycle.

For functional simulation using Quartus II Simulator:

1. A Vector Waveform File (VWF) is created via File > New > Verification/Debugging Files > University Program VWF.
2. Signals clk, reset, pc[31:0], instr[31:0], aluout[31:0], zero are added using the Node Finder (Filter: Pins: All).
3. The clock is set to 10 ns period. Reset is driven high for 0–20 ns then low.
4. The functional simulation netlist is generated via Processing > Generate Functional Simulation Netlist.
5. Simulation is started via Processing > Start Simulation.

5.3 Simulation Results Analysis

The simulation confirms correct operation of all instruction types. Key observations from the waveform:

- Clock cycle 1 (PC=0): add instruction — ALU output = 17 (0x00000011), zero=0. Write to $t2 on next rising edge.
- Clock cycle 2 (PC=4): sub instruction — ALU computes 7−3=4 using 2's complement. ALU output = 4, zero=0.
- Clock cycle 3 (PC=8): and instruction — ALU output = 5 & 12 = 4 (0b0101 & 0b1100 = 0b0100).
- Clock cycle 4 (PC=12): or instruction — ALU output = 5 | 12 = 13 (0b1101).
- Clock cycle 5 (PC=16): slt instruction — 7 is NOT less than 3, so result=0, zero=1.
- Clock cycle 6 (PC=20): lw instruction — memory address = $t0+4 = 9, reads Mem[9].
- Clock cycle 7 (PC=24): sw instruction — Mem[5] is written with $s0=4. MemWrite=1.
- Clock cycle 8 (PC=28): beq instruction — $t0==$t0 so zero=1, branch taken. PC jumps to 44.

The one-clock-cycle execution model is confirmed: each instruction completes within exactly one clock cycle, with register writes on the rising edge following the combinational computation.

6. Making Pin Assignments

Pin assignments specify which physical I/O pins on the FPGA device correspond to each signal in the design. Correct pin assignments are essential for physically implementing the circuit on a development board. Quartus II provides three methods for making pin assignments.

Method 1: Assignment Editor

The Assignment Editor is accessed via Assignments > Assignment Editor. In the Category field, "Pin" was selected. For each signal, the physical pin location is assigned by double-clicking in the Location column. The clock signal (clk) must be assigned to a dedicated clock pin (e.g., PIN_Y2 on the DE2 board) to ensure low-skew distribution through the global clock network.

Method 2: Pin Planner

The Pin Planner provides a graphical view of the FPGA package (FBGA-780 for EP4CE115F29C7) and allows drag-and-drop pin assignment. It is accessed via Assignments > Pin Planner. The device package diagram shows all 529 I/O pins organized by I/O banks.

Method 3: CSV File Import

Pin assignments can also be imported from a CSV file using Assignments > Import Assignments. For the Altera DE2 development board, a pre-made pin assignment file (DE2_pin_assignments.csv) is available from the Altera website.

6.1 Pin Assignment Table for MIPS Processor

The following table shows representative pin assignments for the MIPS processor top-level interface on the Cyclone IV E (EP4CE115F29C7) device, targeting the DE2-115 development board:

Signal	Direction	Pin No.	I/O Bank	I/O Standard
clk	Input	PIN_Y2	Bank 1 (Ded. Clk)	3.3-V LVTTL
reset	Input	PIN_M23	Bank 3	3.3-V LVTTL
pc[0]	Output	PIN_G18	Bank 4	3.3-V LVTTL
pc[1]	Output	PIN_F22	Bank 4	3.3-V LVTTL
pc[2]	Output	PIN_E17	Bank 4	3.3-V LVTTL
writedata[0..7]	Output	PIN_H15–H22	Bank 4	3.3-V LVTTL
dataadr[0..7]	Output	PIN_G15–G22	Bank 4	3.3-V LVTTL
memwrite	Output	PIN_J15	Bank 4	3.3-V LVTTL

All input/output pins are assigned to 3.3-V LVTTL I/O standard. The clock (clk) is assigned to a Dedicated Clock pin (PIN_Y2 on DE2-115) to ensure it is routed through the global clock network, providing low-skew distribution to all flip-flops throughout the design.

The I/O Assignment Warnings shown in the Compilation Report ("Incomplete set of assignments" for all pins) are expected when no manual pin constraints are applied — the

Fitter automatically assigns arbitrary pins. This warning is resolved by making explicit pin assignments via the Assignment Editor.

7. Physically Implementing the Circuit

NOTE: This section is OPTIONAL and was not performed, as no physical FPGA development board was available in this lab environment.

If a Cyclone IV E development board (such as the Altera DE2-115 board) were available, the MIPS processor circuit would be physically programmed onto the FPGA using the following steps:

1. Connect the DE2-115 development board to the computer via USB-Blaster cable and power it on.
2. In Quartus II, open the Programmer by selecting Tools > Programmer (or clicking the Programmer icon in the toolbar).
3. The Programmer window shows Hardware Setup as “USB-Blaster [USB-0]”, Mode as “JTAG”, and the programming file as mips_processor.sof (SRAM Object File) with the Program/Configure checkbox checked.
4. Click Start to program the device. The progress bar reaches 100% and a success message “Configuration Succeeded” appears.
5. After programming, the clock can be driven by the 50 MHz crystal oscillator on the DE2-115 board (PIN_Y2). The reset signal is mapped to a pushbutton (KEY[0]).
6. The processor outputs (PC address, ALU result, memory write data) can be monitored on the seven-segment displays or LEDs of the DE2-115 board.

The compiled programming file mips_processor.sof was generated successfully in the mips/db/ directory during the full compilation flow, confirming that the design is physically implementable on the Cyclone IV E target device.

8. Interpreting the Fitter Equations

The Fitter Equations represent the actual logic circuit implemented by Quartus II on the FPGA device. These equations describe exactly how the logic elements (LEs) and flip-flops in the Cyclone IV E FPGA are configured to realize the MIPS processor design. They can be viewed in the Compilation Report under Fitter > Equations.

8.1 Logic Operators Used in Fitter Equations

The Fitter equations use the following operator symbols common to Altera's equation format:

Symbol	Operation	Description
!	NOT	Logical complement/inversion
&	AND	Logical AND of two signals
#	OR	Logical OR of two signals
$	XOR	Logical exclusive-OR

8.2 Flip-Flop Primitives in the MIPS Processor

The MIPS processor contains three types of registered elements, each inferred from clocked always blocks in the SystemVerilog code:

- PC Register (32 D flip-flops): The PC always @(posedge clk) block infers 32 DFFs for the program counter. Each flip-flop uses the DFFEAS primitive with asynchronous reset (CLRN).
- Register File (1,024 D flip-flops): The 32×32-bit register file always @(posedge clk) write block infers 1,024 DFFs. Register $zero is not synthesized (it is always zero and optimized away).
- Data Memory (2,048 D flip-flops): The 64×32-bit dmem always @(posedge clk) block infers 2,048 DFFs for data storage.

8.3 Example Fitter Equations for the MIPS Processor

After successful compilation, the Fitter equations for the PC register appear as follows. Each bit of the 32-bit PC is implemented as a separate DFFEAS (D flip-flop with synchronous/asynchronous controls):

```
; Fitter Equations for PC register (pc[31:0])
; Each bit is a DFFEAS with GLOBAL clock and active-low reset
pc[0]  = DFFEAS(pcnext[0],  GLOBAL(clk), !reset, VCC, VCC, GND, GND, GND, GND)
pc[1]  = DFFEAS(pcnext[1],  GLOBAL(clk), !reset, VCC, VCC, GND, GND, GND, GND)
...
pc[31] = DFFEAS(pcnext[31], GLOBAL(clk), !reset, VCC, VCC, GND, GND, GND, GND)
```

The ALU's combinational logic appears as LUT (look-up table) equations. For example, the zero flag, which checks if all 32 result bits are zero, is implemented as a tree of OR gates:

```
; Zero flag: NOR reduction of all 32 ALU result bits
zero = !(result[0] # result[1] # result[2] # ... # result[31])
```

The 2's complement subtraction in the ALU generates equations where operand B is conditionally inverted. When ALUcontrol[2]=1, each bit of B is XOR'd with 1 (inversion), and the carry-in to the adder is also set to 1:

```
; SUB implementation: b_mux[i] = ALUcontrol[2] $ b[i]  (XOR = conditional invert)
b_mux[0]  = ALUcontrol[2] $ b[0]
b_mux[1]  = ALUcontrol[2] $ b[1]
; ... carry-chain adder with cin = ALUcontrol[2] implements 2's complement
subtraction
```

These equations confirm that the Fitter has correctly translated the SystemVerilog behavioral description into a functionally equivalent gate-level implementation on the Cyclone IV E FPGA fabric, using the carry-chain primitives native to the device for efficient arithmetic operations.

References

[1] Patterson, D.A. and Hennessy, J.L. (2014). Computer Organization and Design: The Hardware/Software Interface, 5th Edition. Morgan Kaufmann Publishers, Waltham, MA. Chapter 4 & Appendix B.

[2] Altera Corporation. (2013). Quartus II Handbook Version 13.1. San Jose, CA: Altera Corporation. Available: http://www.altera.com/literature/hb/qts/quartusii_handbook.pdf

[3] Altera Corporation. (2013). Altera Quartus II Tutorial — Appendix B: Altera Quartus II CAD System. Included with Quartus II Web Edition 13.1.

[4] IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language. IEEE Std 1800-2012. New York: IEEE, 2012.

[5] Altera Corporation. (2013). Cyclone IV Device Handbook, Volume 1: Cyclone IV E Device Family Data Sheet. San Jose, CA: Altera Corporation.